

Project Author

Name: Waleed Ahmed

GitHub: <https://github.com/Waleedcoderarch/waleed.git>

1. Description

1.1 What is This Project?

This project is a Heart Disease Prediction System. It helps patients and doctors find out whether a person has heart disease or not, just by entering some basic health details into a form on the website. The app takes the details from the user and uses a machine learning model to give the result instantly.

Heart disease is one of the most common and serious health problems in the world. Many people do not know they have it until it is too late. This app makes it easy for anyone to check their risk in seconds without going to a doctor first.

1.2 How Does the App Work?

When a user opens the website, they will see a form. They fill in health details like age, cholesterol, blood pressure and other values. Once they click Predict, the app shows the result — whether the person has heart disease or not, along with a severity percentage.

- The user opens the app in their browser
- The browser sends the request to the ALB (Application Load Balancer) on port 80
- ALB routes traffic to the Kubernetes ClusterIP service
- The service forwards the request to the Flask app on port 5000
- Flask receives form data (age=55, sex=1, cholesterol=300 etc.)
- Flask passes data directly to the ML model — both live in the same container, no extra port needed
- The ML model predicts the result in 0.003 seconds
- Flask sends the result back through the same path to the browser
- The user sees the prediction on screen

1.3 Input Features

The user fills in 13 health details in the form:

Feature	Range / Values	What it means
Age	29 to 77	Age of the patient
Sex	0=Female, 1=Male	Gender of the patient

Feature	Range / Values	What it means
Chest Pain Type	0 to 3	Type of chest pain experienced
Resting BP	94 to 200	Resting blood pressure (mmHg)
Cholesterol	126 to 564	Cholesterol level (mg/dL)
Fasting Blood Sugar	0 or 1	Is blood sugar > 120 mg/dL?
Resting ECG	0 to 2	Resting ECG result
Max Heart Rate	71 to 202	Maximum heart rate achieved (bpm)
Exercise Angina	0 or 1	Chest pain during exercise?
ST Depression	0 to 6.2	ST depression during exercise
Slope	0 to 2	Slope of peak exercise ST segment
Major Vessels	0 to 3	Number of major vessels
Thal	1, 2 or 3	1=Normal, 2=Fixed defect, 3=Reversible

1.4 Dataset

- 1,025 medical records from the UCI Heart Disease Dataset
- Data combined from multiple hospitals
- Target variable: 0 = No heart disease, 1 = Heart disease present
- Data cleaned — removed invalid values in ca and thal columns

1.5 Machine Learning Model

Compared 6 different algorithms. Gradient Boosting gave the best result — 100% accuracy with prediction time of only 0.003 seconds.

Algorithm	Accuracy	Remarks
Gradient Boosting	100%	Selected as final model ✓
Random Forest	High	Slightly lower accuracy
Decision Tree	Medium	Tends to overfit
Logistic Regression	Medium	Linear, misses patterns
Naive Bayes	Medium	Assumes independence
Stacking Classifier	High	More complex, less gain

1.6 Frontend

The frontend is a single HTML page built using HTML and CSS with a dark medical luxury design. The colour theme uses deep black backgrounds with crimson red accents to give it a professional healthcare feel. The page is fully responsive and works on both desktop and mobile screens.

Backend

The frontend is a single HTML page with a dark medical design built using HTML and CSS. The backend is Flask (Python). Flask receives form data, converts it to a pandas DataFrame, passes it to the loaded ML model, gets the prediction probability, and returns the result to the HTML page .

2. Architecture

The diagram below shows the complete end-to-end flow — from developer writing code, to the user getting a prediction in the browser.

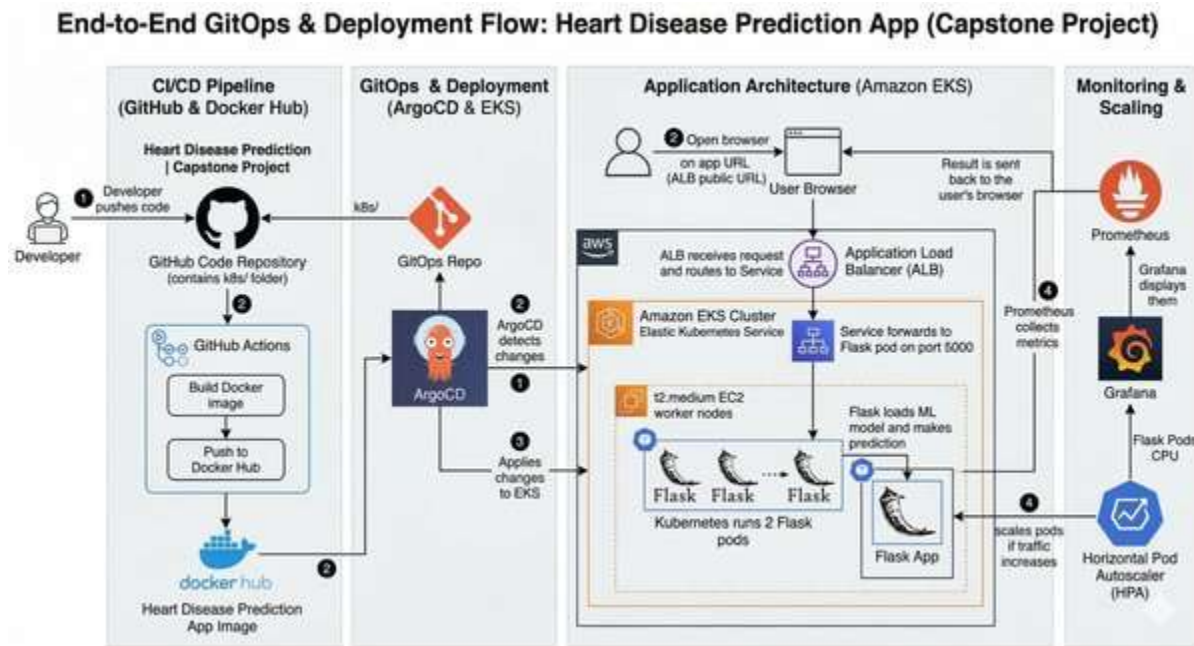


Figure 2.1 — End-to-End GitOps & Deployment Flow: Heart Disease Prediction App

2.1 Flow Summary

- Developer pushes code to GitHub → GitHub Actions builds and pushes Docker image to Docker Hub
- ArgoCD detects k8s/ folder changes on GitHub → auto applies to EKS cluster
- Kubernetes runs 2 Flask pods on t2.medium EC2 worker nodes inside AWS EKS
- User opens app URL (ALB public URL) → ALB receives request on port 80
- ALB routes to ClusterIP Service → Service forwards to Flask pod on port 5000
- Flask loads ML model, makes prediction, result sent back to browser
- HPA monitors CPU → scales pods from 2 to 10 if traffic increases
- Prometheus collects metrics → Grafana displays dashboards

3. Components and Services Used

3.1 Docker

Docker packages the entire application — the Flask code, ML model, all Python libraries and HTML templates — into one container. The app runs exactly the same way on any machine.

- Packages app, ML model and all dependencies in one place
- Solves the works on my machine problem
- Easy to deploy on Kubernetes
- Image stored on Docker Hub: waleedcodearch/heart-disease:latest

3.2 GitHub Actions — CI/CD Pipeline

An automated pipeline that builds and pushes a new Docker image to Docker Hub every time I push code to the main branch. Nothing is done manually.

- Trigger: Push to main branch on GitHub
- Step 1 — Checkout code from GitHub
- Step 2 — Login to Docker Hub using saved secrets
- Step 3 — Build Docker image from the Dockerfile
- Step 4 — Push image to Docker Hub

3.3 Kubernetes Objects

deployment.yaml

Runs 2 replicas of the Flask app with CPU and memory limits. imagePullPolicy: Always ensures the latest image is always pulled from Docker Hub.

service.yaml

Creates a ClusterIP service — internal only, not accessible from the internet. Listens on port 80 and forwards traffic to Flask on port 5000.

ingress.yaml

Tells the AWS Load Balancer Controller to create a public ALB. These three annotations make it happen:

- `kubernetes.io/ingress.class: alb`
- `alb.ingress.kubernetes.io/scheme: internet-facing`
- `alb.ingress.kubernetes.io/target-type: ip`

hpa.yaml

Horizontal Pod Autoscaler — scales pods between 2 and 10 based on CPU. Scale up triggers when CPU goes above 50%.

3.4 AWS Infrastructure

AWS Service	What it does
EKS Cluster	Runs the Kubernetes cluster. AWS manages the master node.
EC2 t2.medium x2	Worker nodes where pods run. Each allows 17 pods max.
ALB	Public entry point. Created automatically by the Ingress object.
ASG	Watches EC2 nodes. If a node crashes, ASG creates a new one.
IAM Policy	Gives ALB controller permission to create and manage load balancers.
OIDC Provider	Trust bridge between EKS and IAM so Kubernetes can use AWS services.
AWS CLI + eksctl	Used from PowerShell to manage the cluster from the command line.

3.5 ArgoCD — GitOps

ArgoCD runs inside Kubernetes and watches the `k8s/` folder in my GitHub repo. When I change a YAML file on GitHub, ArgoCD automatically applies those changes to the cluster — no manual `kubectl` apply needed. Git is the single source of truth.

- Watches GitHub repo every 3 minutes
- Auto applies any changes in `k8s/` folder to EKS
- Shows Synced or OutOfSync status on dashboard
- Tracks all changes — who changed what and when

3.6 Prometheus and Grafana — Monitoring

Installed using Helm (kube-prometheus-stack). Prometheus collects CPU, memory and network metrics from all pods and nodes. Grafana displays all data in real-time dashboards.

- Metrics Server feeds live data to HPA and kubectl top
- ServiceMonitor tells Prometheus which services to scrape
- Grafana shows node health, pod count, CPU, memory, network

3.7 HPA — Horizontal Pod Autoscaler

HPA automatically increases or decreases running pods based on CPU load. I tested this with a busybox load generator that sent continuous traffic. Pods scaled from 2 to 10 automatically. When traffic stopped, pods scaled back down to 2 automatically.

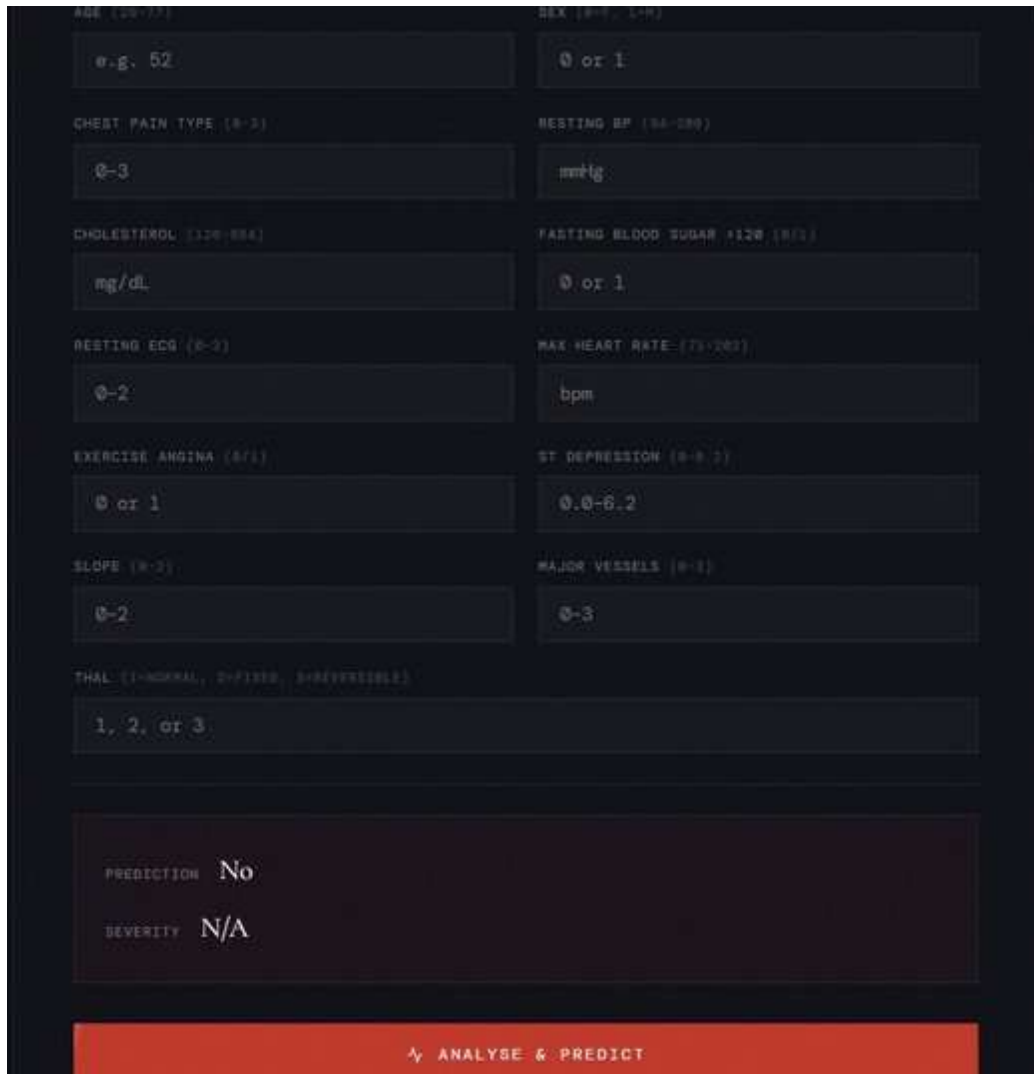
3.8 GitHub Repository

All code is at github.com/Waleedcoderarch/waleed and contains:

- app.py, templates/index.html, gradient_boosting_model.joblib, Dockerfile, requirements.txt
- .github/workflows/docker.yml — CI/CD pipeline
- k8s/ — deployment, service, ingress, hpa, servicemonitor YAML files

4. Screenshots

4.1 Application — Form and Prediction



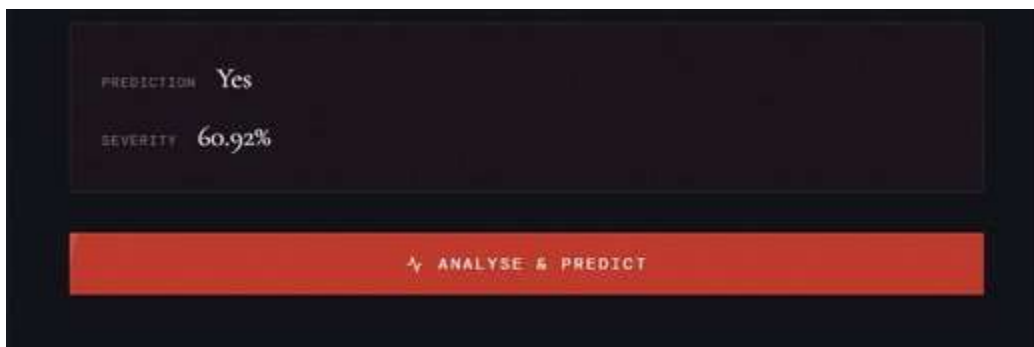
A screenshot of a web application form for heart disease prediction. The form is organized into two columns of input fields. The first column contains: AGE (0-77) with a value of 52; CHEST PAIN TYPE (0-3) with a value of 0-3; CHOLESTEROL (120-684) with a value of mg/dL; RESTING ECG (0-2) with a value of 0-2; EXERCISE ANGINA (0-1) with a value of 0 or 1; SLOPE (0-2) with a value of 0-2; and THAL (1-NORMAL, 0-FIXED, 0-REVERSIBLE) with a value of 1, 2, or 3. The second column contains: SEX (0-F, 1-M) with a value of 0 or 1; RESTING BP (90-200) with a value of mmHg; FASTING BLOOD SUGAR (120-180) with a value of 0 or 1; MAX HEART RATE (70-200) with a value of bpm; ST DEPRESSION (0-0.2) with a value of 0.0-0.2; and MAJOR VESSELS (0-3) with a value of 0-3. Below the input fields, there is a section for the prediction results: PREDICTION No and SEVERITY N/A. At the bottom of the form is a red button labeled ANALYSE & PREDICT.

Field	Value
AGE (0-77)	52
SEX (0-F, 1-M)	0 or 1
CHEST PAIN TYPE (0-3)	0-3
RESTING BP (90-200)	mmHg
CHOLESTEROL (120-684)	mg/dL
FASTING BLOOD SUGAR (120-180)	0 or 1
RESTING ECG (0-2)	0-2
MAX HEART RATE (70-200)	bpm
EXERCISE ANGINA (0-1)	0 or 1
ST DEPRESSION (0-0.2)	0.0-0.2
SLOPE (0-2)	0-2
MAJOR VESSELS (0-3)	0-3
THAL (1-NORMAL, 0-FIXED, 0-REVERSIBLE)	1, 2, or 3

PREDICTION No
SEVERITY N/A

ANALYSE & PREDICT

Figure 4.1 — Heart Disease Prediction Form



A screenshot of the prediction result section of the application. It shows a dark background with white text. The prediction result is displayed as PREDICTION Yes and SEVERITY 60.92%. Below this, there is a red button labeled ANALYSE & PREDICT.

PREDICTION Yes
SEVERITY 60.92%

ANALYSE & PREDICT

Figure 4.2 — Prediction Result: Yes with 60.92% Severity

4.2 Docker Hub



Figure 4.3 — Docker Hub: waleedcodearch/heart-disease (1.9 GB, 246 pulls)

4.3 CI/CD Pipeline — GitHub Actions YAML

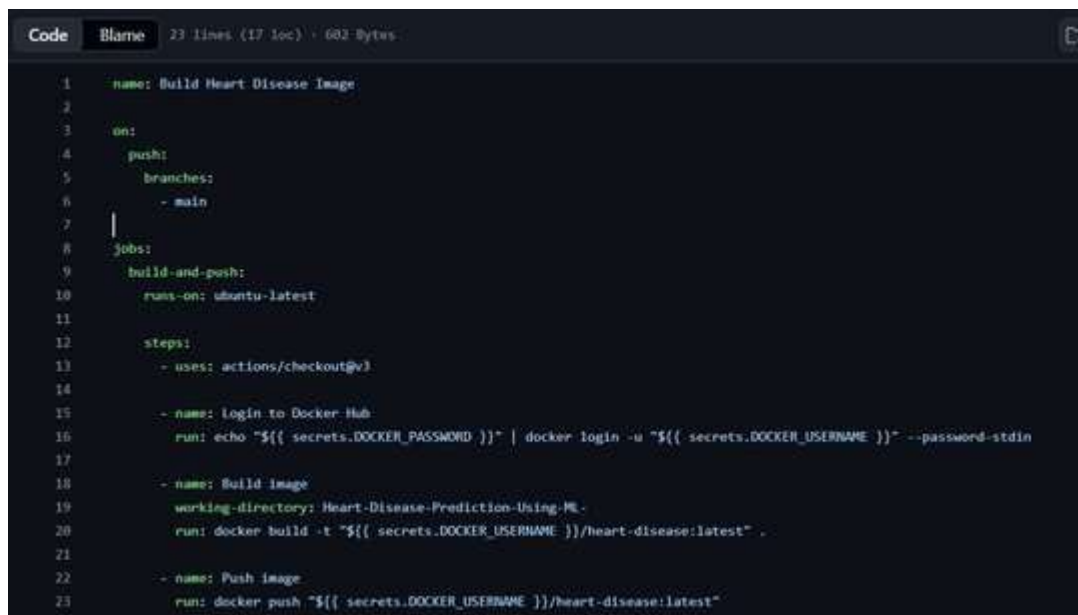


Figure 4.4 — GitHub Actions Pipeline (docker.yml)

4.4 AWS EKS Cluster



Cluster name	Status	Kubernetes version	Support period	Upgrade policy	Created
heart-disease-cluster	Active	1.34 Upgrade now	Standard support until December 2, 2026	Extended support	March 13, 2026, 22:38 (UTC+0)

Figure 4.5 — AWS EKS: heart-disease-cluster (Active, Kubernetes 1.34)

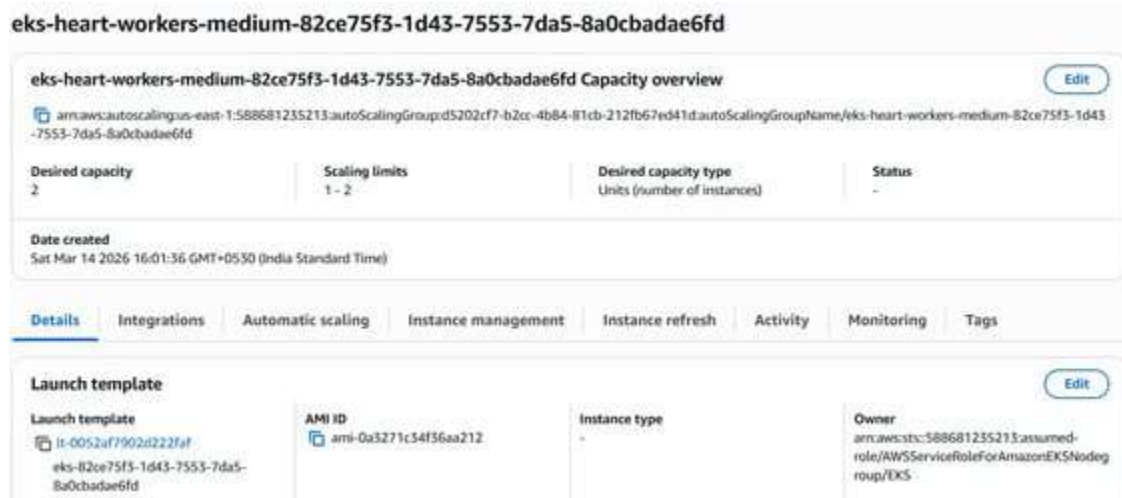
4.5 AWS Application Load Balancer



Details	
Load balancer type Application	Status Active
Scheme Internet-facing	Hosted zone Z35SKDTRQ7X7K
Load balancer ARN arn:aws:elasticloadbalancing:us-east-1:588681235213:loadbalancer/app/k8s-default-heartpre-92a06f80bf/7ad552389ca06466	VPC vpc-060d61f74852770c1
	Availability Zones subnet-07f40f190241a529f us-east-1b (use1-az2) subnet-0357277aa38957cf3a us-east-1f (use1-az5)
	Load balancer IP address type IPv4
	Date created March 13, 2026, 25:43 (UTC+05:30)
	DNS name info k8s-default-heartpre-92a06f80bf-135427511.us-east-1.elb.amazonaws.com (A Record)

Figure 4.6 — AWS ALB: Application type, Internet-facing, Active

4.6 AWS Auto Scaling Group



eks-heart-workers-medium-82ce75f3-1d43-7553-7da5-8a0cbadae6fd Capacity overview			
arn:aws:autoscaling:us-east-1:588681235213:autoScalingGroup:d5202cf7-b2cc-4b84-ff1cb-212fb67ed41d:autoScalingGroupName/eks-heart-workers-medium-82ce75f3-1d43-7553-7da5-8a0cbadae6fd			
Desired capacity 2	Scaling limits 1 - 2	Desired capacity type Units (number of instances)	Status -
Date created Sat Mar 14 2026 16:01:36 GMT+05:30 (India Standard Time)			
Details Integrations Automatic scaling Instance management Instance refresh Activity Monitoring Tags			
Launch template			
Launch template lt-0052af7902d222faf eks-82ce75f3-1d43-7553-7da5-8a0cbadae6fd	AMI ID ami-0a3271c34f36aa212	Instance type -	Owner arn:aws:sts::588681235213:assumed-role/AWSServiceRoleForAmazonEKSCluster/eks-heart-workers-medium-82ce75f3-1d43-7553-7da5-8a0cbadae6fd

Figure 4.7 — ASG: 2 nodes, Desired capacity 2, Scaling limits 1-2

4.7 ArgoCD Dashboard:

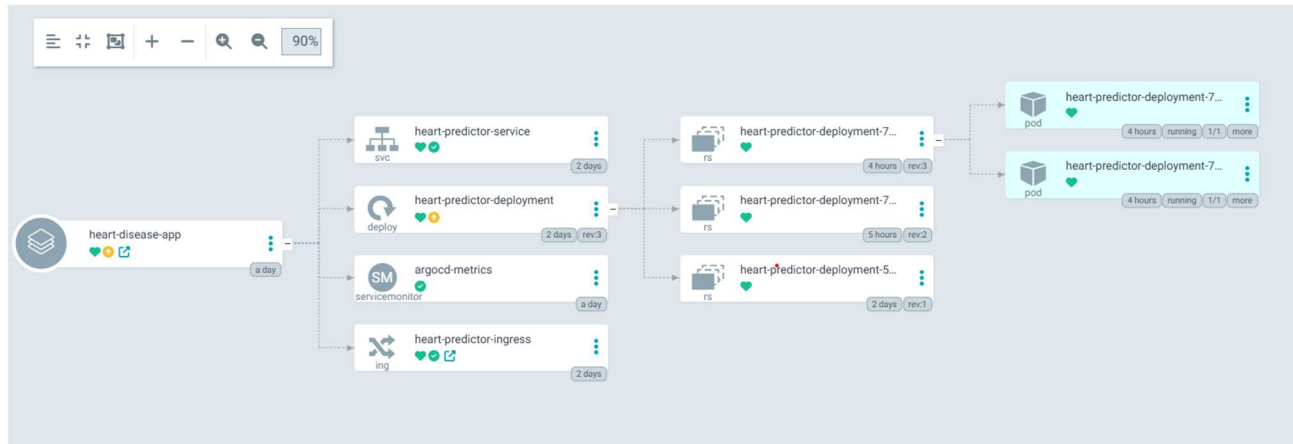


Figure 4.8 — ArgoCD: heart-disease-app with service, deployment, ingress and pods

4.8 Grafana Monitoring Dashboards:

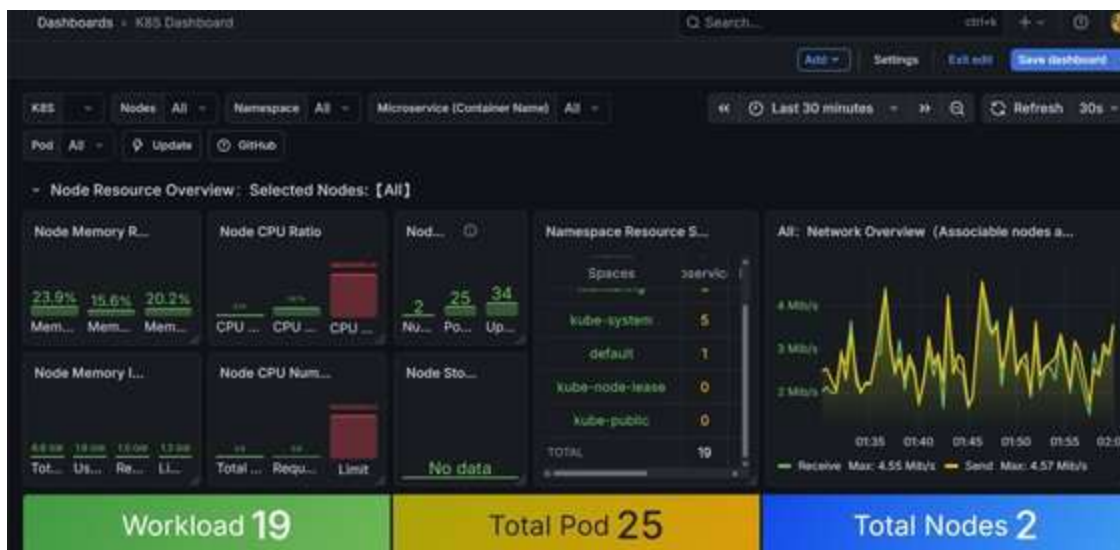


Figure 4.9 — Grafana K8S Dashboard: 2 Nodes, 25 Pods, 19 Workloads

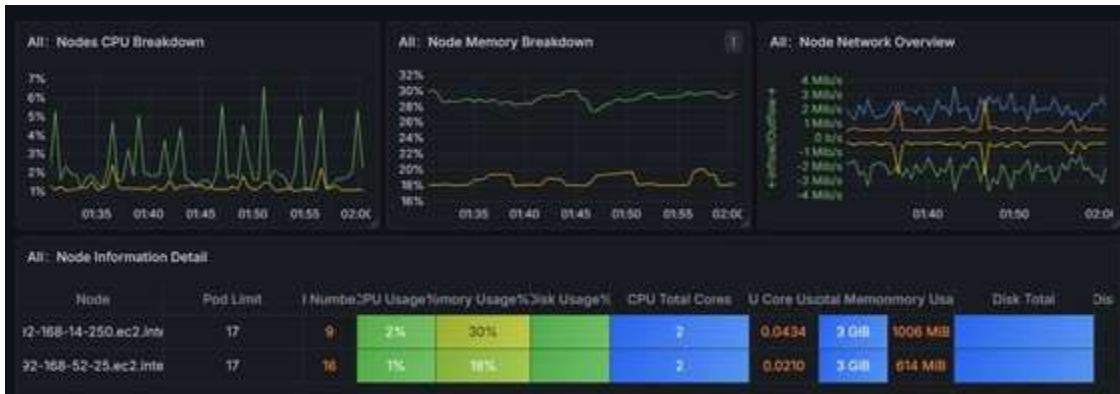


Figure 4.10 — Grafana Node Detail: CPU, Memory and Network Breakdown



Figure 4.11 — Grafana Pod Metrics: CPU, Memory and Network per container